1. What is hoisting?

Hoisting basically means we can use the variable and function before declaring it.

In hoisting variables and functions are hoisted which means their declaration is moved at the top of code.

For Example:

```
console.log(a);
var a = 12;
```

Output: undefined.

What exactly happened here?

The code got converted something like this:

```
var a;
console.log(a);
a = 12;
```

Basically in js, all the declarations move at the top of code and initially its value is undefined.

2. Types in js?

Primitive and Reference

Primitives = number, null, string, symbol, boolean, bigint, undefined(nnssbbu)

Reference - [], (), {}

Aisa koi bhi value jisko copy karne par real copy nhi hota, balki us main value ka reference pass ho jaata hai, use hum reference value kehte hai.

Example:

```
var a = [1,2,3];
var b = a;
```

Yaha b ke liye nai copy nhi bani balki b a ko point kr rha hai. Iska mtlb agar koi changes b ke through kiye jai to wo a me bhi reflect honge.

For Example:

```
b.pop();
console.log(b)    // [1,2]
console.log(a)    // [1,2]
```

Aur jiska copy karen par naya copy bane use primitive kehte hai.

Example:

```
var a = 12;
var b = a;
b = b - 2;

console.log(a);  // 12
console.log(b);  // 10
```

How to actually copy the reference value?

Using the spread operator(...)

```
var a = [1,2,3];
var b = [...a];
```

Here, we have actually the copy and any changes made to b will not effect a.

3. Arguments Vs Parameters

Arguments are passed while calling the function.

Parameters are used in function or you can say recieved.

```
function abcd(a, b) {
      // Here a and b are parameters.
}

abcd(10, 12) // 10, 12 are arguments.
```

4. Arrays

```
      var arr = [1,2,3,4,5,6,7,8]
      arr.push(9);            // [1,2,3,4,5,6,7,8,9]
      arr.pop();        // [1,2,3,4,5,6,7,8]
      arr.unshift(0);         // [0,1,2,3,4,5,6,7,8]
      arr.shift();            // [1,2,3,4,5,6,7,8]
      arr.splice(2, 3); // [1,2,6,7,8]    From 2nd index elements are deleted
and 3 length is deleted.

5. var(ES5) vs let vs const(ES6)
      var                                                          let
                            const

      1. Function Scoped.                                                  1. Braces
Scoped.
         var can be used anywhere in its parent function.
The outer console.log(i) will not give error
         Example:                                                            in
this case.
                  function abcd() {
                      for(var i = 1; i < 12; i++) {
                        console.log(i);
                      }
                      console.log(i);     // It will not give an error
                  }

      2. var adds itself to the window object.                             2. let
and const doesnot add to the window object.

6. What is window?
      There are various features in JavaScript is not actually the part of JS
but we can use it inside JS.
      It uses from window. Window is a box of features given by browser to use
with JS.

      Window is an object given by browser. It is a normal object and contain
various properties and methods.
      It also contains the 'document' of DOM.

      In DOM we create tree like structure. Top level is window -> document ->
html -> head and body. Every node is treated like object.

      In DOM we can perform following:
            1. Access Elements
                  document.getElementById("");  tag, class, querySelector,
querySelectorAll()

            2. Changes
                  First access then make changes.
                  let div = document.querySelector('div');
                  div.textContent = "Hello harry!";

                  Similarly, we have innerHTML, innerText.  getAttributes() and
setAttributes()

            3. Insert Element
                  Two step:
                        1. Create
                              let el = document.createElement("div");

                        2. Add
                              Add using append, prepend, before, after
                              node.append(el);  // need to access the node using
first step(access elements)
```

4. Remove Element

```
let div = document.querySelector('div');
div.remove();
```

Similarly, we have classList to add and remove classes using js.

7. What is Execution Context?

When the JavaScript engine scans a script file, it makes an environment called the Execution Context that handles the entire transformation and execution of the code.

During the context runtime, the parser parses the source code and allocates memory for the variables and functions. The source code is generated and gets executed.

There are two types of execution contexts: global and function. The global execution context is created when a JavaScript script first starts to run, and it represents the global scope in JavaScript. A function execution context is created whenever a function is called, representing the function's local scope.

Phases of the JavaScript Execution Context

There are two phases of JavaScript execution context:

Creation phase: In this phase, the JavaScript engine creates the execution context and sets up the script's environment. It determines the values of variables and functions and sets up the scope chain for the execution context.

Execution phase: In this phase, the JavaScript engine executes the code in the execution context. It processes any statements or expressions in the script and evaluates any function calls.

Everything in JS happens inside this execution context. It is divided into two components. One is memory and the other is code. It is important to remember that these phases and components are applicable to both global and functional execution contexts.

8. Lexical Scope

When we have a nested group of functions, the inner function has access to variables and other resources of the parent function. This means child functions are lexically bound to the execution context of their parents.

Example:

```
function findMyName() {
    var name = "ABD";

    return function printMyName() {
        console.log(name);
    }
}
```

The printMyName() function had access to name variable of findMyName() function. This will work only in one direction, you cannot access variables and other resources residing in the inner function from the parent function.

9. Lexical Enviroment

When the JavaScript engine creates a new execution context for a function, it creates a new lexical environment to store variables defined in that function during the execution phase.

A lexical Environment is a data structure that holds an identifier-variable mapping. (here identifier refers to the name of variables/functions, and the variable is the reference to the actual object [including function type object] or primitive value).

The lexical environment contains two components:

Environment record: It is the actual place where the variable and function declarations are stored.
Reference to the outer environment: It means it has access to its outer (parent) lexical environment.

8. Falsy values = 0, false, undefined, null, NaN, document.all
Truthy values = Everything other than falsy.

9. forEach: forEach is used to loop the array. It doesnot change the original array.

```
var a = [1,2,3,4,5,6,7,8,9];
a.forEach(function(val) {
   console.log(val + 2);     // 3,4,5,6,7,8,9,10,11
})
```

10. forIn: forIn is used to loop object.

```
var obj = {
    name: "ABD",
    age: 21,
    city: "Bhopal"
}

   for(var key in obj) {
      console.log(key, obj[key]);   // name ABD       age 21
 city Bhopal
   }
```

forOf loop is used to loop through the iterable(Array or String)

11. CallBack Functions
CallBack Functions are the functions which are not executed directly and are executed only when certain actions are performed or certain other function is executed.
For Example:

```
setTimeout(function() {
            console.log("2 second baad chala");
 }, 2000);
```

12. First Class Functions
A programming language is said to have First-class functions if functions in that language are treated like other variables. So the functions can be assigned to any other variable or passed as an argument or can be returned by another function.

```
function abcd(a) {
   a();           // Hey
}

abcd(function() {console.log("Hey");}
```

13. How array is created behind the scene?
Array is created like objects behind the scene.

```
var a = [100,22,38,4,5];
// Internally its something like this:
  var a = {
      0: 100,
      1: 22,
      2: 38,
      3: 4,
      4: 5
  }
```

Although, type of [] and type of {} is object.

```
Array.isArray([])    // true
Array.isArray({})    // false
```

Since, array is an object we can create negative indexes in js.

```
var arr = [1,2,3];
arr[-1] = 5;   // No Error
```

14. How to delete props from obj.

```
delete obj.prop_name;
```

15. Operators and Expression

A fragment of code that produces a value is called an expression. Every value written literally is an expression.

For example: 7 or Harry

Arithmetic(+,-,*,/,**,%,++,--), Assignment, Comparator(==, !=, ===, !==, >, <, >=, <=, ?), logical, bitwise

== compares only value

=== compares value as well as type

16.

```
let marks = {
  harry: 90,
  shubham: 9,
  lavish: 56,
  monika: 4
}

for(let i = 0; i < Object.keys(marks).length; i++) {
  console.log(Object.keys(marks)[i]);
}
```

Object.keys(marks) gives the array.

```
for(let key in marks) {
  console.log(key + " " + marks[key]);
}
```

17. Template Literals(``) is also used to make strings. You can use variables directly in template literal. This is called string interpolation.

You can use single quotations or double quotations inside template literal.

```
let a = 10;
console.log(`Value of a is ${a}`);
```

18. Escape Sequence Characters

Because strings must be written within quotes, JavaScript will misunderstand this string:

```
let text = "We are the so-called "Vikings" from the north.";
```

The string will be chopped to "We are the so-called ".

The solution to avoid this problem, is to use the backslash escape character.

The backslash (\) escape character turns special characters into string characters:

Code	Result	Description
\'	'	Single quote
\"	"	Double quote
\\	\	Backslash

19. Strings are immutable. Whenever you apply any methods on String it returns the new string. Original string is not changed.

20. Array methods
      push, pop, shift, unshift, delete(delete the element but length does not decreases)
      concat(can concat more than two arrays) [Does not change the original array]
      sort[sorts alpahbetically not numerically, to sort numerically you should make compare function and return (a - b)]00
      splice(changes original array and returns the deleted elements) and slice(doesnot change original array)

21. for each, for of and for in in array
      let num = [1,2,3,4,5]
      for each: to loop through array
                  num.foreach((element) => { console.log(element)})   // 1 2 3 4 5

      for in: for(let i in num) {console.log(i)}   // 0 1 2 3 4 It print keys which are basically index in array

      for of: for(let i of num) {console.log(i)}   // 1 2 3 4 5

      Array.form: convert object or string in array
                  let str = "Hello"
                  let arr = Array.form(str)
                  console.log(arr)

22. map, filter and reduce
      map: creates a new array by performing some operations on each array element.
            const a = [1,2,3]
            a.map(((value, index, array)=>{return value*value;)

      filter: filter array on some condition and returns new array. Doesnot modify original array

      reduce: reduces an array to single value.

      let arr = [21,45,10, 4, 2]

      let a1 = arr.map((value, index) => { return value + index; }
      console.log(a1); // [22, 46, 12, 7, 6]

      let a2 = arr.filter((val) => { return val < 10;)   // returns new array
      console.log(a2)   // [4,2]

      let a3 = arr.reduce((val1, val2)=>{return val1 + val2)})
      console.log(a3)   // 82(summation)

      Difference between foreach and map?
      map creates new array whereas foreach doesnot creates new array

23. Advantages of seperate script file
      Seperation of concerns
      Browser Caching

24. Console methods
      console.log():   To print message
      console.error(): To show error
      console.assert(55>45): If statement is correct, no problem otherwise error
      console.clear(): To clear console
      console.table(): To show key value pairs in table
      console.warn():   To show warning

console.info(): Similar to console.log. It is just shown in info section rather than in message section
console.time(): To note the starting time
console.timeEnd(): To note the ending time

By using time and timeEnd, you can find how much time it takes to run a code.

25. alert, prompt, confirm
alert: Used to invoke a mini window with a tag
prompt: Used to take user input as string
confirm: shows a message and waits for the user to press ok or cancel. Returns true for ok and false for cancel.

26. What is the DOM?
The Document Object Model (DOM) is a programming interface for web documents. It represents the page so that programs can change the document structure, style, and content. The DOM represents the document as nodes and objects; that way, programming languages can interact with the page.

27. What is BOM?
The BOM represents additional objects provided by the browser(host enviroment) for working with everyting except the document.

The functions alert, prompt and confirm are also the parts of BOM.

28. We can access the elements using dom using different methods.
For example: document.body.firstChild, lastChild, childNodes.
childNodes doesnot return an array it returns nodeList. To convert it into array you need to use Array.form()

```
let a = Array.from(document.body.childNodes)
```